Endpoint Network Connectivity Troubleshooting Toolkit (DNS, Proxy, VPN, IP/Port/URL Filtering)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary 4

The problem . . . 4

The solution . . . 4

What's inside . . . 4

Why it matters . . . 4

Bottom line . . . 5

Solution Overview 5

The problem: "it works on my machine" . . . 5

The endpoint-to-service path . . . 5

The layered root-cause model . . . 5

What "vendor-neutral" means here . . . 6

Where the scripts fit . . . 6

Requirements 7

R1. Access to the affected endpoint . . . 7

R2. A known-good control endpoint . . . 7

R3. The exact test targets (name, IP, port, URL) . . . 8

R4. Knowledge of the expected network path . . . 8

R5. PowerShell 5.1+ (Windows PowerShell) on the endpoint . . . 8

R6. Microsoft Graph PowerShell module (only for scope targeting) . . . 8

R7. Permission to run reset actions (only for remediation) . . . 8

Diagnostic Playbook 9

Step 0 — Baseline and control (always do this first) . . . 9

The symptom tree . . . 9

Branch A — Name won't resolve (DNS, layer 1) . . . 10

Branch B — Resolves but wrong proxy / PAC misroute (layer 2) . . . 10

Branch C — VPN / split-tunnel gap (layer 3) . . . 11

Branch D — IP:Port blocked (firewall, layer 4) . . . 12

Branch E — URL / category blocked (web filter, layer 5) . . . 12

After you isolate: safe local remediation . . . 13

Rolling it up across many targets . . . 13

Testing 13

Gate 0 — Control comparison (isolation gate) . . . 14

Gate 1 — DNS resolution . . . 14

Gate 2 — Proxy / PAC routing . . . 14

Gate 3 — VPN / split-tunnel routing . . . 14

Gate 4 — IP : Port reachability . . . 15
Gate 5 — URL / category access . . . 15
Bulk gate — coverage across a target list . . . 15
Evidence to keep from a test run . . . 16
Troubleshooting 16
A. Tooling and method issues . . . 16
B. Symptom → layer quick lookup . . . 16
C. When the evidence points off the endpoint . . . 17
Best Practices 17
1. Always work top-down through the layers . . . 17
2. Change one thing, test one thing, prove one layer . . . 17
3. Always keep a control . . . 18
4. Prove the layer before you escalate . . . 18
5. Capture evidence as you go, not after . . . 18
6. Distinguish a drop from a refusal . . . 18
7. Know the intended path before judging the actual one . . . 18
8. Prefer safe, local, reversible remediation — and only for local causes . . . 18
9. Defer vendor console internals to the vendor . . . 18
10. Pilot before fleet-wide validation . . . 18
11. Build and reuse a target library . . . 19
12. Re-baseline after any network change . . . 19
Deliverables Summary 19
Documentation . . . 19
PowerShell scripts . . . 20
How the pieces work together . . . 20
Appendix A — PowerShell Scripts 21

Executive Summary

The problem

Connectivity failures are among the most expensive and frustrating incidents in any managed environment. A user "can't reach" an application, website, or cloud service; an identical machine beside them works fine. The failure could be **DNS**, a **proxy or PAC misroute**, a **VPN split-tunnel gap**, an **IP/port firewall block**, or a **URL/category filter** — and without a method, teams *guess*: reboot, reinstall, blame the network. Guesswork burns hours, escalates the wrong tickets to the wrong teams, and lets the same root cause recur.

The solution

This toolkit replaces guessing with a **vendor-neutral, endpoint-side, layered root-cause method**. It models every connection as a stack of layers — DNS → proxy/PAC → VPN → firewall → URL filter — and gives you the **exact check that proves each layer** from the affected Windows endpoint. You isolate the failure to **one** layer, prove it with saved evidence, and either fix it locally or escalate to the owning team with proof instead of a hunch.

What's inside

- A **Solution Overview** and layered model explaining the endpoint-to-service path.
- A **Requirements Guide** (endpoint access, a control device, exact targets, expected path).
- A **Diagnostic Playbook** — a symptom tree with per-layer checks for every common failure mode.
- **Five PowerShell scripts** — posture discovery, per-target reachability validation, a coverage report, safe local remediation (default preview mode), and a pilot-group builder.
- **Testing** (pass/fail gates that name the responsible layer), **Troubleshooting**, and **Best Practices** for repeatable, evidence-based triage.

Why it matters

- **Faster resolution.** A defined top-down method finds the blocking layer in minutes, not a day of trial and error.
- **Right team, first time.** Evidence-backed escalation ("port to the target times out; control host on the same port succeeds") is actioned immediately instead of bouncing between teams.
- **Repeatable and reusable.** The same scripts and target lists apply to every connectivity incident and every new app rollout — you keep them and reuse them.
- **Safe.** Discovery, validation, and reporting are read-only; remediation and scope changes preview by default and only touch safe, local, reversible settings.

Bottom line

The exact layer blocking the endpoint — DNS, proxy, VPN, or a filter — proven with a repeatable method and scripts you keep and reuse on every connectivity incident. Vendor-neutral, self-guided, tested, and a fraction of the cost of ad-hoc consulting.

Solution Overview

The problem: "it works on my machine"

A user reports that an application, website, or cloud service "won't load," "times out," or "can't connect." One endpoint fails while an identical one nearby works fine. The instinct is to guess — restart the app, reboot, blame the Wi-Fi — and hope. Guessing wastes time and, worse, hides the real cause so it comes back tomorrow.

This solution replaces guessing with a **repeatable, layered root-cause method**. It is **vendor-neutral** and **endpoint-side**: everything is tested and proven from the affected Windows device (or from a comparable known-good device beside it), so you isolate exactly which layer is blocking the traffic before you escalate to a network, proxy, VPN, or firewall team.

The endpoint-to-service path

Every connection from a managed Windows endpoint to a service crosses a **stack of layers**. Traffic must succeed at each one, in order, or the connection fails. When something "can't connect," the block lives in exactly one of these layers — your job is to find which one:

```
[ Endpoint app ]
      |  1. Name resolution (DNS)          "what IP is this name?"
      v
[ Resolver / DNS server ]
      |  2. Proxy decision (PAC / system proxy) "do I go direct or through a proxy?"
      v
[ Proxy (explicit or PAC-selected) ]        (may be bypassed for some destinations)
      |  3. VPN / tunnel routing            "is this destination inside the tunnel or split out?"
      v
[ VPN client / split-tunnel rules ]
      |  4. IP + Port reachability          "can a TCP/UDP session open to host:port?"
      v
[ Firewall / IP + Port filter ]
      |  5. URL / category filter           "is this URL or category allowed?"
      v
[ URL / web filter ]
      v
[ Service ]  <- success
```

Read it top to bottom. A failure at an **earlier** layer masks everything below it: if a name won't resolve (layer 1), you never even get to test the port (layer 4). That is why the method always works **from the top down** — resolve the name first, then route, then reach the port, then the URL.

The layered root-cause model

Each layer has a distinct **question**, a distinct **symptom** when it fails, and a distinct **check** that proves it. This is the heart of the toolkit:

#	Layer	Question it answers	Typical failure symptom	The check that proves it
1	**DNS (name resolution)**	What IP does this name map to?	"Name won't resolve" / NXDOMAIN / wrong IP	`Resolve-DnsName`, `nslookup` against the configured and a known-good resolver
2	**Proxy / PAC**	Direct, or through a proxy? Which one?	Connects for some sites, not others; wrong route	Read system + WinHTTP proxy, evaluate the PAC verdict for the URL
3	**VPN / split-tunnel**	Is the destination inside or outside the tunnel?	Works off-VPN but not on (or vice-versa); internal-only apps unreachable	Compare routing table and interface with VPN up vs down
4	**IP : Port (firewall)**	Can a TCP/UDP session open to host:port?	Name resolves but connection times out / refused	`Test-NetConnection -Port`, raw TCP connect to the resolved IP
5	**URL / category filter**	Is this URL or web category allowed?	Port opens but request returns a block page / 403 / reset mid-TLS	Fetch the URL and inspect the response vs a known-allowed URL

The single most important habit: isolate one layer at a time, top to bottom, and **prove** the verdict with a check whose output you can save as evidence. When you can say "the name resolved to `203.0.113.10`, the TCP session to `203.0.113.10:443` refused, but the same port to a control host succeeded," you have not guessed — you have proven the block is at the firewall/port layer and you can escalate with proof.

What "vendor-neutral" means here

The blocking device at layer 2, 3, 4, or 5 might be any vendor's proxy, VPN concentrator, firewall, or secure web gateway. This toolkit does **not** tell you which buttons to press in a specific vendor's console — those screens change and differ per product. Instead it proves, from the endpoint, **which layer** is responsible, so you hand a precise, evidence-backed finding to whoever owns that device. Where a vendor console is genuinely required (for example, to see why a URL category is blocked or to read a proxy's own logs), the playbook says so and defers to **that vendor's official documentation and your own console**.

Where the scripts fit

Five PowerShell scripts operationalize the method so you run the same checks the same way every time:

- **NETDX-Discovery.ps1** — captures the endpoint's connectivity posture: DNS servers, system and

WinHTTP proxy, PAC URL, VPN/interface state, and default routes. Read-only.

• **NETDX-Validation.ps1** — for a list of target endpoints, tests each layer per target: name resolves, IP:port reachable, URL returns an allowed response. Read-only.

• **NETDX-Report.ps1** — rolls validation results into a connectivity-coverage report (which targets pass, which fail, and at which layer). Read-only.

• **NETDX-Remediation.ps1** — applies **safe, local, reversible** resets (flush DNS cache, reset route cache, clear a stale WinHTTP proxy) with `-WhatIf` on by default. Never touches network gear.

• **NETDX-Set-ScopeTargeting.ps1** — builds/maintains a pilot Entra group for staged testing via Microsoft Graph, `-WhatIf` on by default.

Use Discovery to see the posture, Validation to test each target, Report to summarize, Remediation only for safe local resets, and Set-ScopeTargeting to scope a pilot before wider testing.

Requirements

Before you run the diagnostic method, confirm the items below. These are **in addition to** the category-level prerequisites in *Global Requirements — Network* (see `global-requirements-network.md`). Each requirement is written as **Description / Why / Verify / Implement if missing** so a competent-but-novice technician can confirm each one by clicking.

> *This is endpoint-side **troubleshooting**. You do not need to own or reconfigure the proxy, VPN, or firewall to use this toolkit — you need enough access to *observe* the endpoint and *know what it should be reaching*. Where a change on network gear is required, this toolkit produces the evidence to hand to that team.*

R1. Access to the affected endpoint

Description: Interactive or remote access to the Windows device that is failing, as a user who can open an elevated PowerShell prompt. **Why:** The whole method proves the block *from the endpoint*. You must run local checks (DNS, proxy, routing, port tests) on the machine that is actually failing — not on your own working machine. **Verify:** Sign in to the device (RDP, Intune remote help, or in person). Right-click **Start** → **Windows PowerShell (Admin)** and confirm it opens with "Administrator" in the title bar. **Implement if missing:** Arrange remote access (Intune → **Devices** → **Remote help**, or your remote support tool). If the user cannot elevate, have them run the read-only Discovery/Validation scripts, which do not require elevation for their core checks.

R2. A known-good control endpoint

Description: A second, comparable device on the same or a similar network that **can** reach the target service. **Why:** Isolation needs a control. "Both fail" points to the service or a shared upstream layer; "only this one fails" points to the endpoint's own config (DNS, proxy, VPN, local firewall). Without a control you cannot separate "the service is down" from "this endpoint is misconfigured." **Verify:** From the control device, confirm it reaches the target service (open the app / URL). **Implement if missing:** Use any nearby working device — a colleague's machine, a spare, or a VM on the same network segment. Note whether it

is on the same subnet/VLAN/VPN state as the failing one.

R3. The exact test targets (name, IP, port, URL)

Description: A written list of what the endpoint is *supposed* to reach: the DNS name(s), the expected IP or IP range, the TCP/UDP port(s), and the full URL(s) of the failing service. **Why:** You cannot test reachability without knowing the destination. "The app is broken" is not a target; `app.example.com`, `443`, `https://app.example.com/health` is. Precise targets make each layer's pass/fail unambiguous. **Verify:** For each service, you can state its FQDN, expected port, and a URL you can request. The `NETDX-Validation.ps1` script consumes exactly this list (a CSV of `Name,Host,Port,Url`). **Implement if missing:** Get the targets from the app vendor's connectivity/firewall documentation, the service's admin portal, or by capturing them on the known-good control endpoint (see the Playbook for how to observe what a working device connects to).

R4. Knowledge of the expected network path

Description: Awareness of whether the endpoint is *supposed* to go **direct**, **through a proxy/PAC**, or **through a VPN tunnel** to reach each target, and the expected proxy host / PAC URL / VPN profile name if any. **Why:** A "wrong route" is only wrong relative to what was intended. To judge the proxy and VPN layers you must know the intended behavior — e.g. "this internal app must go through the VPN tunnel" or "internet traffic should egress via the corporate proxy `your-proxy-host:8080`." **Verify:** You can state, per target, the intended path (direct / proxy / VPN) and the expected proxy or VPN identifiers. Discovery reports the *actual* posture so you can compare. **Implement if missing:** Ask the network/proxy/VPN owner for the intended egress design, or read the managed proxy/VPN configuration profile pushed by Intune (Intune → **Devices** → **Configuration**).

R5. PowerShell 5.1+ (Windows PowerShell) on the endpoint

Description: Windows PowerShell 5.1 (built into Windows 10/11) or PowerShell 7+, able to run the included scripts. **Why:** The scripts use standard cmdlets (`Resolve-DnsName`, `Test-NetConnection`, `Get-NetRoute`, `netsh`) available on managed Windows. **Verify:** In PowerShell run `$PSVersionTable.PSVersion` and confirm the major version is 5 or higher. **Implement if missing:** Windows PowerShell 5.1 ships with Windows; if it is somehow absent, install it via **Settings** → **Apps** → **Optional features**, or install PowerShell 7 from the Microsoft Store.

R6. Microsoft Graph PowerShell module (only for scope targeting)

Description: The `Microsoft.Graph` PowerShell module, needed **only** for `NETDX-Set-ScopeTargeting.ps1` (building a pilot Entra group). **Why:** Scope targeting creates/updates an Entra security group of pilot devices via Graph; the other four scripts do not need it. **Verify:** Run `Get-Module Microsoft.Graph -ListAvailable`. If it returns nothing, install it. **Implement if missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`. The scope-targeting script requests the `Group.ReadWrite.All` and `GroupMember.ReadWrite.All` delegated scopes at sign-in.

R7. Permission to run reset actions (only for remediation)

Description: Local admin rights on the endpoint and organizational approval to run **safe, local** resets (flush DNS cache, reset route cache, clear a stale local proxy setting). **Why:** `NETDX-Remediation.ps1` performs only local, reversible actions, but they still touch the endpoint's networking state, so they run with `-WhatIf` on by default and should be approved. **Verify:** You can open an elevated PowerShell prompt (see R1) and change control permits a local reset on the affected device. **Implement if missing:** Obtain local admin and a change approval, or keep the remediation script in `-WhatIf` (preview) mode and hand the previewed actions to whoever is authorized.

Diagnostic Playbook

This is the working method: a **symptom tree** that isolates a connectivity failure to exactly one layer, with the **exact checks per layer** and how to read each result. Work **top to bottom** — an earlier layer failing masks everything below it. At every step, **save the output as evidence**; the goal is not just to fix it but to *prove* which layer is responsible.

> ***Golden rule:*** *change one thing, test one thing, prove one layer. Never restart the app and reboot and swap the cable all at once — you learn nothing.*

Step 0 — Baseline and control (always do this first)

1. On the **failing** endpoint, open **Windows PowerShell (Admin)**.
2. Capture the posture: run `.\NETDX-Discovery.ps1`. This records DNS servers, system + WinHTTP proxy,

PAC URL, VPN/interface state, and default routes into a JSON/CSV file you keep as evidence.

1. On the **known-good control** endpoint (R2), open the same target service and confirm it works.
2. Decide the split:

• **Control also fails** → the problem is upstream/shared or the service itself. Skip endpoint

config; escalate with the target list and the fact that multiple endpoints fail.

• **Only the failing endpoint fails** → continue down the tree; the block is in this endpoint's

path.

The symptom tree

Pick the branch that matches what you observe, then run that layer's checks.

```
Can't reach the service?
■■ Does the NAME resolve?                          ■■ NO ■■■ Branch A: DNS
■    ■ YES
■    ▼
■■ Is the route/proxy/VPN decision correct?
■    ■■ Wrong proxy / PAC verdict?                ■■■■■■■ Branch B: Proxy / PAC
■    ■■ On VPN but internal target unreachable?   ■■■■■■■ Branch C: VPN / split-tunnel
■    ■ route looks right
■    ▼
■■ Does IP:PORT open (name resolved, still no connect)? NO ■■ Branch D: IP / Port (firewall)
■    ■ YES
■    ▼
■■ Does the URL return an ALLOWED response?        ■■ NO ■■■ Branch E: URL / category filter
```

Branch A — Name won't resolve (DNS, layer 1)

Symptom: "Server not found," NXDOMAIN, `Resolve-DnsName` errors, or the name resolves to an obviously wrong IP (e.g. a captive-portal or sinkhole address).

Checks (run on the failing endpoint):

1. Resolve against the **configured** resolver:

`Resolve-DnsName app.example.com` (replace with your target FQDN).

1. Resolve against a **known-good public** resolver to compare:

`Resolve-DnsName app.example.com -Server 1.1.1.1`

1. See which resolvers the endpoint is actually using:

`Get-DnsClientServerAddress -AddressFamily IPv4`

1. Check for a stale/poisoned cache or a hosts-file override:

`Get-DnsClientCache | Where-Object Entry -like "*example.com*"` and open `C:\Windows\System32\drivers\etc\hosts`.

Read the result:

• Configured resolver fails **but** public resolver succeeds → the endpoint's DNS server (or a DNS-layer filter) is the block. Evidence: the two differing outputs.

• Both fail → the name genuinely does not resolve anywhere (typo, decommissioned host, or the authoritative zone is down) — verify the FQDN with the app owner.

• Resolves to a wrong/sinkhole IP → a DNS filter or hosts entry is redirecting it; note the returned IP as evidence.

• Stale cache entry → see remediation (safe local flush) below.

Prove & escalate: name resolution is layer 1; if it fails only via the configured resolver, hand the two `Resolve-DnsName` outputs to whoever owns DNS/DNS-filtering.

Branch B — Resolves but wrong proxy / PAC misroute (layer 2)

Symptom: Some sites work and others don't; a site that should go direct is trying a proxy (or vice versa); intermittent depending on network; `407 Proxy Authentication Required`.

Checks (on the failing endpoint):

1. System (WinINET) proxy for the user:

`Get-ItemProperty 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Internet Settings'` (look at `ProxyEnable`, `ProxyServer`, `AutoConfigURL`).

1. WinHTTP proxy (used by services and many system components):

`netsh winhttp show proxy`

1. If a **PAC** is in use, fetch and read it:

download the `AutoConfigURL` and inspect the `FindProxyForURL` logic for your target host — does it return `DIRECT` or a proxy for that URL?

1. Compare against the **intended** path from R4.

Read the result:

• A proxy is set that should not be (or a stale `ProxyServer` remains) → the endpoint is misrouting;

this is a layer-2 block. A safe local clear of a stale WinHTTP proxy is in remediation.

• PAC returns the wrong verdict for the URL → the PAC file (owned by the proxy/network team) is

misrouting; capture the URL and the PAC's verdict and escalate to the PAC owner.

• `407` → the proxy is in the path and demanding auth the endpoint isn't supplying; that is expected

proxy behavior — confirm credentials/SSO with the proxy owner (defer to that vendor's console).

Prove & escalate: capture `netsh winhttp show proxy`, the WinINET values, and the PAC verdict for the specific URL. That trio proves whether the endpoint is choosing the wrong path.

Branch C — VPN / split-tunnel gap (layer 3)

Symptom: Internal-only apps are unreachable while on VPN (or *only* reachable on VPN); works at the office but not remote; some corporate ranges route out to the internet instead of the tunnel.

Checks (on the failing endpoint, VPN connected):

1. Confirm the VPN interface is up:

`Get-NetAdapter | Where-Object Status -eq 'Up'` and `Get-NetIPInterface -ConnectionState Connected`.

1. Look at the route for the target IP (from Branch A's resolved IP):

`Find-NetRoute -RemoteIPAddress 10.20.30.40` — which interface does it choose, the VPN or the physical NIC?

1. List the full routing table to see split-tunnel scope:

`Get-NetRoute -AddressFamily IPv4 | Sort-Object -Property RouteMetric`.

1. Repeat step 2 **with the VPN disconnected** to compare.

Read the result:

• Target IP routes out the **physical NIC** while on VPN, but the app is internal → it is a

split-tunnel gap: that destination range is not included in the tunnel. This is a layer-3 config on the VPN profile (owned by the VPN team). Capture the two `Find-NetRoute` outputs (VPN up vs down) as proof.

• VPN interface is **not up** at all → the tunnel didn't establish; that is a VPN-client problem, not

routing — defer to your VPN vendor's client documentation.

• Target routes correctly through the tunnel but still fails → the block is *below* VPN; drop to

Branch D.

Prove & escalate: the pair of `Find-NetRoute` results (tunnel up vs down) plus the target IP proves whether the split-tunnel scope omits the destination.

Branch D — IP:Port blocked (firewall, layer 4)

Symptom: The name resolves (Branch A passed) and the route looks right, but the connection **times out** (silent drop) or is **refused** (RST). "Connecting…" that never completes.

Checks (on the failing endpoint):

1. Test the exact host and port:

```
Test-NetConnection app.example.com -Port 443 -InformationLevel Detailed
```

1. Test the **resolved IP** directly (removes DNS from the equation):

```
Test-NetConnection 203.0.113.10 -Port 443
```

1. Test the **same port to a control host** you know is open, to prove it isn't a local block:

e.g. `Test-NetConnection www.microsoft.com -Port 443`

1. Rule out the **local** Windows firewall:

`Get-NetFirewallProfile | Select-Object Name, Enabled` and check for a rule blocking the app (`Get-NetFirewallRule -Enabled True -Direction Outbound`).

Read the result:

• `TcpTestSucceeded : False` with a **timeout** → traffic is being **silently dropped** — classic network firewall / IP filter block between the endpoint and the service. A drop (no response) points upstream, not to the local host.

• **Refused** (fast RST) → something answered and rejected — often the service isn't listening on that port, or a device is actively resetting. Confirm the port with the app owner.

• Control host on the **same port succeeds** but the target fails → the port itself isn't locally blocked; the block is specific to the target's IP/port at an upstream firewall/IP filter.

• Local Windows firewall has a matching outbound block rule → that is the block, and it's local (fixable on the endpoint per your policy).

Prove & escalate: the three `Test-NetConnection` results (target host, target IP, control host) together prove the block is at the IP:port layer and whether it is local or upstream. Hand them to the firewall owner (defer to that vendor's console for the specific rule).

Branch E — URL / category blocked (web filter, layer 5)

Symptom: The port opens (Branch D passed) but the request returns a **block page**, a `403`, a redirect to a filtering portal, or the TLS session is **reset mid-handshake**. Only *certain URLs* on an otherwise-reachable host fail.

Checks (on the failing endpoint):

1. Request the URL and inspect the response:

`Invoke-WebRequest https://app.example.com/health -UseBasicParsing -MaximumRedirection 0` — note the status code, any redirect `Location`, and whether the body is a filter block page.

1. Compare with a **known-allowed** URL on a benign category to confirm general web access works.

2. Inspect the returned certificate / TLS behavior: an unexpected certificate issuer on the response can indicate an inspecting/blocking web gateway in the path.

1. If your organization runs a secure web gateway or URL filter, note the **block/observed category** shown on the block page (it usually names the category and a request ID).

Read the result:

• Returns a **filter block page / 403 / redirect to a portal** → a URL or category filter is blocking it (layer 5). Capture the status code, the block-page text, and any request/transaction ID.

• **TLS reset mid-handshake** for one host but not others → a filtering/inspection device is terminating that specific destination.

• Known-allowed control URL also fails → the problem is broader than one URL (drop back up the tree).

Prove & escalate: the block-page text (with category + request ID) plus the passing control URL proves a category/URL policy is responsible. Escalate to whoever owns the web filter / secure web gateway and **defer to that vendor's console** to see and adjust the category policy.

After you isolate: safe local remediation

Only for blocks that are **local to the endpoint** (stale DNS cache, stale route cache, stale WinHTTP proxy) run `.\NETDX-Remediation.ps1`. It defaults to `-WhatIf` (preview only). Review the previewed actions, then re-run with `-WhatIf:$false` (and, if your policy requires, `-Confirm`) to apply. It never touches proxy, VPN, or firewall hardware — those are escalated with the evidence you captured.

Rolling it up across many targets

To test a whole connectivity list at once (e.g. all the endpoints a new SaaS app needs), feed the R3 target CSV to `.\NETDX-Validation.ps1`, then summarize with `.\NETDX-Report.ps1` for a pass/fail-by-layer coverage view — the same method, applied in bulk.

Testing

Testing here is not a separate phase after a "deployment" — it *is* the method. Each gate below is a **pass/fail check that maps to one layer** of the endpoint-to-service path. Run them top to bottom; the first gate that fails names the responsible layer and points you to the matching branch of the Diagnostic Playbook. Record every result as evidence.

> ***How to read this:*** *a **PASS** means that layer is not the block — proceed down. The **first FAIL** is your root cause layer. You should never have two layers "failing" at once from a single test run, because an*

upstream fail prevents the downstream check from being meaningful.

Gate 0 — Control comparison (isolation gate)

Check	Same target reached from the known-good control endpoint (R2).
PASS	Control reaches it; failing endpoint does not → problem is endpoint-local, continue.
FAIL	Control **also** fails → problem is shared/upstream or the service; stop endpoint triage, escalate with the target list.
Layer	Not a single layer — this gate decides *whether* to walk the endpoint's stack at all.

Gate 1 — DNS resolution

Check	`Resolve-DnsName <FQDN>` (configured resolver) and `-Server 1.1.1.1` (control resolver).
PASS	Both resolve to the same, correct IP.
FAIL	Configured resolver errors/NXDOMAINs or returns a wrong/sinkhole IP while the control resolver succeeds.
Responsible layer	**Layer 1 — DNS.** Go to Playbook Branch A.

Gate 2 — Proxy / PAC routing

Check	`netsh winhttp show proxy` + WinINET settings + PAC verdict for the URL, compared to the intended path (R4).
PASS	The endpoint's proxy decision for the target matches intent (correct proxy, or correctly `DIRECT`).
FAIL	A stale/wrong proxy is set, or the PAC returns the wrong verdict, or a `407` blocks the request.
Responsible layer	**Layer 2 — Proxy / PAC.** Go to Playbook Branch B.

Gate 3 — VPN / split-tunnel routing

Check	`Find-NetRoute -RemoteIPAddress <resolved IP>` with VPN up vs down; confirm VPN interface state.
PASS	The target IP routes through the intended interface (tunnel for internal, physical for direct).

FAIL	An internal target routes out the physical NIC (split-tunnel gap) or the VPN interface is not up.
Responsible layer	**Layer 3 — VPN / split-tunnel.** Go to Playbook Branch C.

Gate 4 — IP : Port reachability

Check	`Test-NetConnection <host> -Port <port>`, the resolved IP directly, and the same port to a control host.
PASS	`TcpTestSucceeded : True` to the target host/IP.
FAIL	Timeout (silent drop → upstream firewall) or refused (RST → nothing listening / active reset), while the control-host port succeeds.
Responsible layer	**Layer 4 — IP / Port firewall.** Go to Playbook Branch D.

Gate 5 — URL / category access

Check	`Invoke-WebRequest <URL>` inspected for status code, redirect, block-page body, and TLS behavior, vs a known-allowed control URL.
PASS	The URL returns the expected allowed response (e.g. 200 / expected redirect), like the control URL.
FAIL	Block page, `403`, redirect to a filtering portal, or mid-TLS reset for that URL while the control URL passes.
Responsible layer	**Layer 5 — URL / category web filter.** Go to Playbook Branch E.

Bulk gate — coverage across a target list

When validating many targets at once (R3 CSV), each row runs Gates 1, 4, and 5 (and reports proxy/VPN posture from Discovery). `NETDX-Report.ps1` summarizes:

- **Overall pass rate** — how many targets are fully reachable.
- **First-failing-layer breakdown** — how many failures are DNS vs port vs URL, so you can see whether

one layer (e.g. a firewall change) explains most failures.

- **Per-target detail** — the exact gate each failing target stopped at.

A clean run is **all targets PASS at every applicable gate**. Any FAIL is actionable: it names the target, the gate, and therefore the layer and the team to escalate to.

Evidence to keep from a test run

For each incident, retain: the Discovery output (posture), the per-gate command output that produced the first FAIL, and the matching control result (control endpoint / control resolver / control host / control URL). That set is the proof that the block is at the named layer.

Troubleshooting

Two kinds of snags: (A) problems with the **method and the tools themselves**, and (B) a fast **symptom → layer** lookup when you just need to jump to the right branch of the Playbook. For the full per-layer checks, see the Diagnostic Playbook; for the pass/fail gates, see Testing.

A. Tooling and method issues

`Resolve-DnsName` **/** `Test-NetConnection` **not recognized.** These ship with Windows PowerShell 5.1+. Confirm with `$PSVersionTable.PSVersion` (see R5). On a minimal/PowerShell-7-only host, `Test-NetConnection` lives in the Windows-only `NetTCPIP` module — run the scripts under Windows PowerShell 5.1 if a cmdlet is missing.

Scripts won't run — "running scripts is disabled on this system." Launch with the documented invocation: `powershell -ExecutionPolicy Bypass -File .\NETDX-Discovery.ps1`. This bypasses policy for that one process only and changes nothing permanently.

Discovery/Validation report needs elevation. The core read-only checks do not require admin, but a few (e.g. reading some WinHTTP state) are richer when elevated. Run from **Windows PowerShell (Admin)** for the fullest posture; otherwise the scripts still run and note any field they could not read.

`NETDX-Set-ScopeTargeting.ps1` **fails at sign-in / "module not found."** Install the Graph module (R6): `Install-Module Microsoft.Graph -Scope CurrentUser`. At run time it requests `Group.ReadWrite.All` and `GroupMember.ReadWrite.All`; an admin may need to consent. If consent is refused, keep it in `-WhatIf` and hand the previewed changes to a Graph-permitted admin.

Remediation "did nothing." It defaults to `-WhatIf` — that is preview mode, by design. Review the previewed actions, then re-run with `-WhatIf:$false` to apply. It only performs safe local resets; if the block is not local (proxy/VPN/firewall), remediation intentionally cannot help — escalate with evidence instead.

Validation shows every target failing at Gate 1 (DNS). Usually the endpoint's own resolver or a DNS-layer filter is down, not every target at once. Re-run a single `Resolve-DnsName <FQDN> -Server 1.1.1.1` against a control resolver; if that succeeds, the configured resolver/DNS filter is the shared cause (Branch A).

Results differ between two runs. Connectivity is stateful — a VPN reconnected, a PAC changed, a cache populated. Re-run Discovery to re-baseline (Best Practice 12) and note the network state (VPN up/down, on-site vs remote) with each run.

B. Symptom → layer quick lookup

You observe…	Likely layer	Go to
"Server not found," NXDOMAIN, or a wrong/sinkhole IP	**1 — DNS**	Playbook Branch A
Some sites work, others don't; `407`; wrong direct/proxy behavior	**2 — Proxy / PAC**	Playbook Branch B
Works off-VPN not on (or vice-versa); internal apps unreachable	**3 — VPN / split-tunnel**	Playbook Branch C
Name resolves but connection **times out** (silent)	**4 — IP:Port (firewall, upstream drop)**	Playbook Branch D
Name resolves but connection is **refused** (fast RST)	**4 — IP:Port (nothing listening / active reset)**	Playbook Branch D
Port opens but you get a block page / `403` / portal redirect	**5 — URL / category filter**	Playbook Branch E
TLS reset mid-handshake for one host only	**5 — inspecting/filtering gateway**	Playbook Branch E
Both the failing endpoint **and** the control fail	shared upstream / service	Gate 0 — escalate with target list

C. When the evidence points off the endpoint

Once a gate proves the block is **not** local (upstream firewall, PAC/proxy owned by the network team, a VPN split-tunnel profile, a URL category policy), the endpoint side is done. Package the captured evidence — Discovery posture, the first-failing-gate output, and the matching control result — and escalate to the owning team. For the specific device's own configuration and logs (proxy, VPN concentrator, firewall, secure web gateway), **defer to that vendor's official documentation and your own console**; this toolkit's job is to prove which layer and which team, and it has done that.

Best Practices

These are the habits that make network triage **repeatable** and escalation **evidence-based**. They apply on every connectivity incident, regardless of vendor.

1. Always work top-down through the layers

Resolve the name first, then the route/proxy/VPN decision, then the port, then the URL. An earlier layer failing masks everything below it, so testing out of order wastes time and produces misleading results. The symptom tree in the Playbook exists precisely to keep you in order.

2. Change one thing, test one thing, prove one layer

Never restart the app, reboot, flush DNS, and toggle the VPN all at once. If it then works you have learned nothing and cannot document a root cause. Isolate a single variable, re-test the single matching gate, and record whether it changed the result.

3. Always keep a control

A control endpoint, a control DNS resolver (a public one), a control host on the same port, and a control URL of a benign category. Every verdict in this method is a *comparison*: "the target fails but the control passes." Without a control you can prove nothing.

4. Prove the layer before you escalate

Do not send "the app is broken" to the network team. Send: "name resolves to `203.0.113.10`; TCP to `203.0.113.10:443` times out; the same port to a control host succeeds — this is an upstream IP:port block." An evidence-backed ticket is actioned in minutes; a vague one bounces for days.

5. Capture evidence as you go, not after

Save the Discovery output at the start and the exact command output for the first failing gate. Screen memory fades and endpoints get rebooted; a saved JSON/text artifact is the durable proof. The scripts write to files for this reason.

6. Distinguish a drop from a refusal

A **timeout** (no response) points upstream — a firewall silently dropping traffic. A **refusal** (fast RST) means something answered and said no — often the service isn't listening on that port, or a device is actively resetting. They lead to different owners; don't conflate them.

7. Know the intended path before judging the actual one

"Wrong proxy" and "split-tunnel gap" only mean something relative to design intent. Confirm from R4 whether each target is *supposed* to go direct, via proxy, or via VPN before you call the actual behavior wrong.

8. Prefer safe, local, reversible remediation — and only for local causes

Flushing the DNS cache, resetting the route cache, or clearing a stale local proxy are safe and reversible, and only make sense when the block is *local to the endpoint*. Never "fix" a proxy, VPN, or firewall from the endpoint — those are escalated with evidence to their owners. Keep remediation in `-WhatIf` until you have confirmed the cause is local.

9. Defer vendor console internals to the vendor

This toolkit proves *which layer* blocks the traffic. Reading a proxy's own logs, adjusting a URL category, or editing a VPN split-tunnel profile happens in that vendor's console — follow that vendor's official documentation and your own console for the specific control.

10. Pilot before fleet-wide validation

When rolling out a new app or a connectivity change, scope a small pilot group first (`NETDX-Set-ScopeTargeting.ps1`), validate the full target list against it, and only then expand. Catching a layer-4 firewall gap on five pilot devices is far cheaper than on the whole fleet.

11. Build and reuse a target library

Keep the R3 target CSVs per application. The next time that app "breaks," you re-run Validation against a known-good target list in seconds instead of rediscovering what it should reach. Treat these lists as reusable assets.

12. Re-baseline after any network change

Proxy, PAC, VPN, and firewall changes drift the endpoint posture. After any such change, re-run Discovery to capture the new baseline so your next incident compares against reality, not a stale snapshot.

Deliverables Summary

Everything included in the **Endpoint Network Connectivity Troubleshooting Toolkit** and how each piece is used.

Documentation

Document	What it gives you
Solution Overview	The endpoint-to-service path (DNS → proxy/PAC → VPN → firewall/URL filter → service) and the layered root-cause model behind the whole method.
Requirements Guide	Each prerequisite as Description / Why / Verify / Implement — endpoint access, a control device, exact test targets, expected path, PowerShell, Graph module, and reset permissions.
Diagnostic Playbook	The symptom tree and the exact per-layer checks: name won't resolve, resolves-can't-connect, proxy/PAC misroute, VPN split-tunnel gap, IP:Port blocked, URL category blocked.
Testing Guide	Pass/fail gates, one per layer, that name the responsible layer on the first failure.
Troubleshooting Guide	Fixes and reads for the method's own snags (the tools themselves, not the network) plus fast layer-by-layer symptom lookups.
Best Practices	The habits for repeatable triage and evidence-based escalation.
Global Requirements — Network	The category-level prerequisites shared by all Network solutions.

PowerShell scripts

All scripts carry a standard header documenting **PURPOSE**, **HOW TO RUN**, and **EXPECTED RESULTS**. Discovery, Validation, and Report are **read-only**. Remediation and Set-ScopeTargeting default to `-WhatIf` (preview) so nothing changes until you explicitly confirm.

Script	Role	Read-only?
NETDX-Discovery.ps1	Capture the endpoint's connectivity posture: DNS servers, system + WinHTTP proxy, PAC URL, VPN/interface state, default routes.	Yes
NETDX-Validation.ps1	For each target in a CSV, test the layers: name resolves, IP:port reachable, URL returns an allowed response.	Yes
NETDX-Report.ps1	Roll validation results into a connectivity-coverage report (pass rate, first-failing-layer breakdown, per-target detail).	Yes
NETDX-Remediation.ps1	Safe, local, reversible resets only: flush DNS cache, reset route cache, clear a stale WinHTTP proxy. Never touches network gear.	No (`-WhatIf` default)
NETDX-Set-ScopeTargeting.ps1	Build/maintain a pilot Entra security group via Microsoft Graph for staged testing.	No (`-WhatIf` default)

How the pieces work together

1. **Discovery** on the failing endpoint records the posture (evidence baseline).
2. The **Playbook** walks the symptom tree; **Testing** gates name the responsible layer.
3. **Validation** + **Report** apply the same checks in bulk across a target list.
4. **Remediation** fixes only local causes, safely and reversibly.
5. **Set-ScopeTargeting** scopes a pilot before fleet-wide validation.

The result of any incident is a proven root-cause layer with saved evidence — ready to fix locally or escalate to the owning team.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

NETDX-Discovery.ps1

Purpose: Records the affected Windows endpoint's network posture so you have an evidence baseline before troubleshooting: configured DNS servers, system (WinINET) and WinHTTP proxy, any PAC URL, VPN/interface state, and default routes. Read-only; changes nothing. Save the output as evidence and compare against a known-good control endpoint. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NETDX-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\NETDX-Discovery.ps1 -OutputPath C:\Temp\netdx (no special scopes; run from an elevated prompt for the fullest posture) EXPECTED RESULTS A console summary plus a JSON file (netdx-discovery-COMPUTER-TIMESTAMP.json) capturing the posture. Admin sees DNS servers, proxy/PAC config, VPN/adapter state, and default routes. Pass = the file is written and the posture matches the intended design; investigate any unexpected proxy, missing VPN interface, or wrong DNS server. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

How to run: powershell -ExecutionPolicy Bypass -File .\NETDX-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\NETDX-Discovery.ps1 -OutputPath C:\Temp\netdx (no special scopes; run from an elevated prompt for the fullest posture) EXPECTED RESULTS A console summary plus a JSON file (netdx-discovery-COMPUTER-TIMESTAMP.json) capturing the posture. Admin sees DNS servers, proxy/PAC config, VPN/adapter state, and default routes. Pass = the file is written and the posture matches the intended design; investigate any unexpected proxy, missing VPN interface, or wrong DNS server. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

NETDX-Remediation.ps1

Purpose: Applies only safe, local, reversible resets on the affected endpoint when the block is proven to be local: flush the DNS client cache, reset the IP route cache, and clear a stale WinHTTP proxy. It never touches proxy servers, VPN concentrators, or firewalls — those are escalated with evidence. Defaults to preview mode (-WhatIf on) so nothing changes until you confirm. Run only after the Diagnostic Playbook proves the cause is local to this endpoint. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NETDX-Remediation.ps1 (preview only — shows what would change) powershell -ExecutionPolicy Bypass -File .\NETDX-Remediation.ps1 -WhatIf:$false (apply; run from an elevated prompt; add -Confirm if your policy requires a prompt) Optional switches: -FlushDns -ResetRouteCache -ClearWinHttpProxy (default: all enabled) EXPECTED RESULTS In preview, admin sees each action prefixed "What if:" and nothing changes. With -WhatIf:$false the selected resets run and a per-action PASS/FAIL is shown. Pass = the selected local resets completed. If the issue persists, the cause is not local — escalate the captured evidence to the owning team. .NOTES White-label — the repeatable way. Remediation defaults to -WhatIf (preview). #>

How to run: powershell -ExecutionPolicy Bypass -File .\NETDX-Remediation.ps1 (preview only — shows what would change) powershell -ExecutionPolicy Bypass -File .\NETDX-Remediation.ps1 -WhatIf:$false (apply; run from an elevated prompt; add -Confirm if your policy requires a prompt) Optional switches: -FlushDns -ResetRouteCache -ClearWinHttpProxy (default: all enabled) EXPECTED RESULTS In preview, admin sees each action prefixed "What if:" and nothing changes. With -WhatIf:$false the selected resets run and a per-action PASS/FAIL is shown. Pass = the selected local resets completed. If the issue persists, the cause is not local — escalate the captured evidence to the owning team. .NOTES White-label

— the repeatable way. Remediation defaults to -WhatIf (preview). #>

NETDX-Report.ps1

Purpose: Reads a NETDX-Validation results file (CSV or JSON) and produces a connectivity-coverage summary: overall pass rate, a breakdown of failures by first-failing layer (DNS / Port / Url), and a per-target detail list. Read-only; summarizes existing results and writes an HTML/CSV report you can attach to a ticket as evidence. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NETDX-Report.ps1 -ResultsFile .\netdx-validation-20260701-120000.csv powershell -ExecutionPolicy Bypass -File .\NETDX-Report.ps1 -ResultsFile .\results.json -OutputPath C:\Temp\netdx (no special scopes) EXPECTED RESULTS A console summary (pass rate, first-failing-layer counts) plus an HTML report (netdx-report-TIMESTAMP.html) and a summary CSV. Pass = all targets reachable (100 percent). Any shortfall lists which targets failed and at which layer, so you can escalate the right layer to the right team. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

How to run: powershell -ExecutionPolicy Bypass -File .\NETDX-Report.ps1 -ResultsFile .\netdx-validation-20260701-120000.csv powershell -ExecutionPolicy Bypass -File .\NETDX-Report.ps1 -ResultsFile .\results.json -OutputPath C:\Temp\netdx (no special scopes) EXPECTED RESULTS A console summary (pass rate, first-failing-layer counts) plus an HTML report (netdx-report-TIMESTAMP.html) and a summary CSV. Pass = all targets reachable (100 percent). Any shortfall lists which targets failed and at which layer, so you can escalate the right layer to the right team. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

NETDX-Set-ScopeTargeting.ps1

Purpose: Creates (if missing) and populates a Microsoft Entra security group of pilot devices via Microsoft Graph so connectivity changes and validation runs are scoped to a small pilot before fleet-wide rollout. Defaults to preview mode (-WhatIf on) so no group or membership changes are made until you confirm. Supply device object IDs (or device names) to add to the pilot. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NETDX-Set-ScopeTargeting.ps1 -GroupName "NETDX-Pilot-Devices" -DeviceIds id1,id2 (preview only) powershell -ExecutionPolicy Bypass -File .\NETDX-Set-ScopeTargeting.ps1 -GroupName "NETDX-Pilot-Devices" -DeviceIds id1,id2 -WhatIf:$false Required Graph scopes: Group.ReadWrite.All, GroupMember.ReadWrite.All Requires the Microsoft.Graph module: Install-Module Microsoft.Graph -Scope CurrentUser EXPECTED RESULTS In preview, admin sees the group that would be created and members that would be added, with no changes. With -WhatIf:$false the group is created if missing and the devices are added. Pass = the pilot group exists and contains the requested devices; use it to scope validation. .NOTES White-label — the repeatable way. Scope targeting defaults to -WhatIf (preview). #>

How to run: powershell -ExecutionPolicy Bypass -File .\NETDX-Set-ScopeTargeting.ps1 -GroupName "NETDX-Pilot-Devices" -DeviceIds id1,id2 (preview only) powershell -ExecutionPolicy Bypass -File .\NETDX-Set-ScopeTargeting.ps1 -GroupName "NETDX-Pilot-Devices" -DeviceIds id1,id2 -WhatIf:$false Required Graph scopes: Group.ReadWrite.All, GroupMember.ReadWrite.All Requires the Microsoft.Graph module: Install-Module Microsoft.Graph -Scope CurrentUser EXPECTED RESULTS In preview, admin sees the group that would be created and members that would be added, with no changes. With -WhatIf:$false the group is created if missing and the devices are added. Pass = the pilot group exists and contains the requested devices; use it to scope validation. .NOTES White-label — the repeatable way. Scope targeting defaults to -WhatIf (preview). #>

NETDX-Validation.ps1

Purpose: For each target in a CSV, tests the layers from the endpoint: does the NAME resolve, does the IP:PORT open, and does the URL return an allowed response. Produces a per-target pass/fail with the first-failing layer so you know which layer blocks each target. Read-only; opens no write operations on the endpoint or the network. CSV columns: Name,Host,Port,Url (Url optional; Port optional, defaults to 443). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NETDX-Validation.ps1 -TargetCsv .\targets.csv powershell -ExecutionPolicy Bypass -File .\NETDX-Validation.ps1 -TargetCsv .\targets.csv -OutputPath C:\Temp\netdx (no special scopes) EXPECTED RESULTS A console table of each target with DNS/Port/Url results and the first failing layer, plus a results file (netdx-validation-TIMESTAMP.json/csv) for NETDX-Report.ps1. Pass = all applicable checks succeed for a target. Any fail names the responsible layer (DNS / Port / Url). .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

How to run: powershell -ExecutionPolicy Bypass -File .\NETDX-Validation.ps1 -TargetCsv .\targets.csv powershell -ExecutionPolicy Bypass -File .\NETDX-Validation.ps1 -TargetCsv .\targets.csv -OutputPath C:\Temp\netdx (no special scopes) EXPECTED RESULTS A console table of each target with DNS/Port/Url results and the first failing layer, plus a results file (netdx-validation-TIMESTAMP.json/csv) for NETDX-Report.ps1. Pass = all applicable checks succeed for a target. Any fail names the responsible layer (DNS / Port / Url). .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>